

Gesture Controlled Robotic Arm

A vision-based, gesture-driven control system for a robotic arm using Google MediaPipe, Arduino Uno and a MeArm — built as Minor Project-1 (22UECL505).

Institution	SDM College of Engineering and Technology, Dharwad
Department	Electronics and Communication Engineering
Course	Minor Project-1 [22UECL505], 5th Semester, Academic Year 2025-26
Guide	Assistant Prof. Raghuram K. M, Dept. of E&CE;
Author	Sourab R (2SD24EC409)

Overview

This project is about controlling a robotic arm using hand gestures instead of a joystick or remote. I used a webcam along with Google's MediaPipe framework to track my hand in real time, and mapped the hand movements to servo commands that control a MeArm robotic arm through an Arduino Uno over Bluetooth.

The idea came from wanting a more natural way to control a robot — instead of learning buttons or writing commands, you just move your hand and the arm follows. It's also a good way to make robots more accessible for people who might find physical controllers difficult to use.

Problem Statement

A few problems with the usual way robotic arms are controlled motivated this project:

- **Complex interfaces** — joysticks and remotes need practice, and programming them isn't beginner-friendly.
 - **Accessibility** — physical controllers can be hard to use for people with mobility issues.
 - **Cost** — proper vision-based industrial systems are expensive for a student or hobby setup.
 - **Flexibility** — fixed control schemes don't adapt well to real-time, dynamic interaction.
-

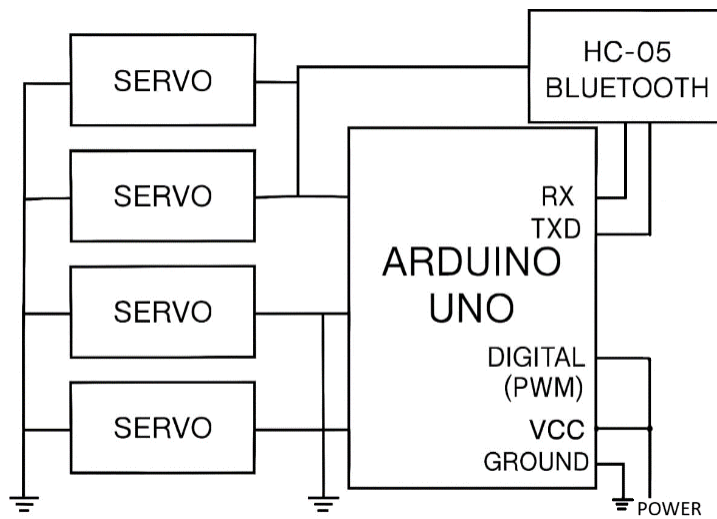
Objectives

- Build a robotic arm that responds to hand gestures in real time using MediaPipe.
 - Get rid of physical controllers and replace them with simple camera-based input.
 - Make the control hands-free so it's easier for people with limited mobility to use.
 - Keep the whole setup affordable enough for a student project.
 - Run everything locally so it doesn't depend on an internet connection.
 - Keep the gesture set flexible so new gestures can be added later.
-

How It's Put Together

Signal Flow

Webcam → MediaPipe hand landmark detection → gesture recognition → mapping coordinates to servo angles → sent over Bluetooth (HC-05) → Arduino Uno → 4 servo motors (X, Y, Z, claw) on the MeArm.



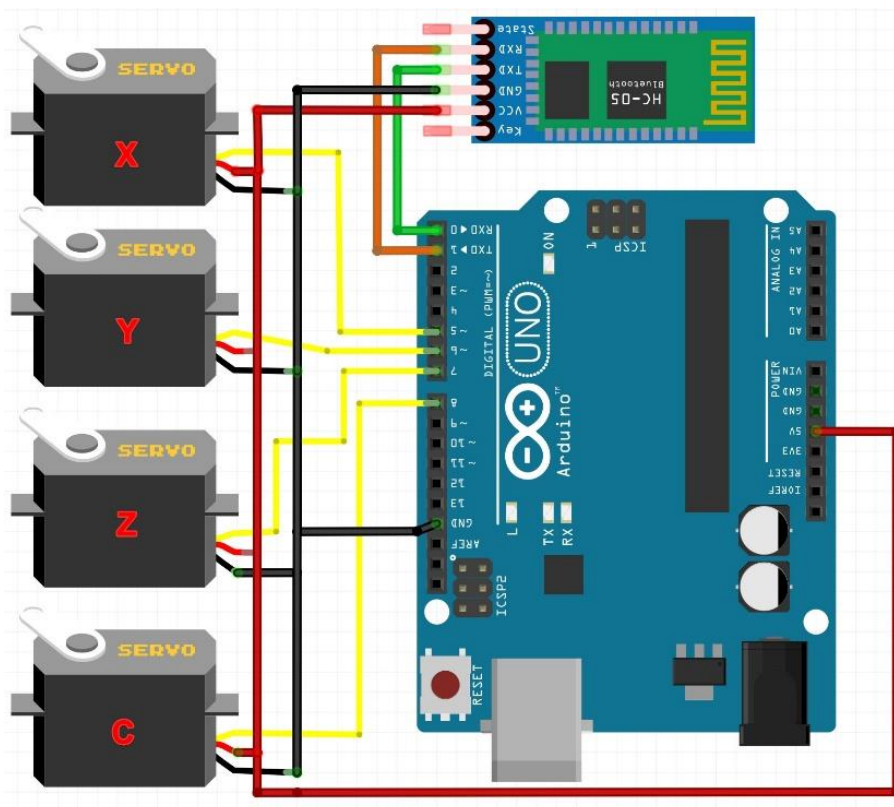
Block diagram of the system

Hardware Used

- MeArm robotic arm (4 servos: X, Y, Z, and claw/gripper)
- Arduino Uno
- HC-05 Bluetooth module for wireless serial communication
- Webcam / phone camera (DroidCam, optional)
- Power supply, cables, connectors and mounting hardware

Software Used

- Python 3
- MediaPipe for hand landmark detection
- OpenCV for video capture and processing
- PySerial for talking to the Arduino over Bluetooth
- Arduino IDE for the servo control firmware



Working Principle

- 1 The webcam continuously captures my hand in the frame.
- 2 MediaPipe Hands processes each frame and extracts 21 (x, y, z) landmark points on the hand.
- 3 Distances between the fingertips and palm base are compared against thresholds to recognize gestures like open palm or fist.
- 4 Hand position is linearly mapped to servo angles, and clamped so it stays within each servo's mechanical limits.
- 5 The computed angles are sent to the Arduino over Bluetooth (HC-05).
- 6 The Arduino drives the four servos — X, Y, Z, and claw — to match, moving the MeArm in real time.

Gesture-to-Action Mapping

Gesture	Robotic Arm Action
Hand position (x, y, z)	Mapped to X / Y / Z servo angles of the arm
Open palm	Claw opens
Fist	Claw closes

Key Features

- **Gesture-based control** — no physical controllers needed, just the camera.
- **Real-time response** — the arm follows my hand with minimal delay.
- **Multi-servo control** — X, Y, Z and the claw all move independently.
- **Wireless** — HC-05 Bluetooth links the PC and Arduino, so no wires between them.
- **Works offline** — MediaPipe runs locally, no internet needed.
- **Low-cost** — just an Arduino Uno, a few servos and a regular webcam.
- **Customizable** — gesture thresholds and mappings can be tweaked easily.

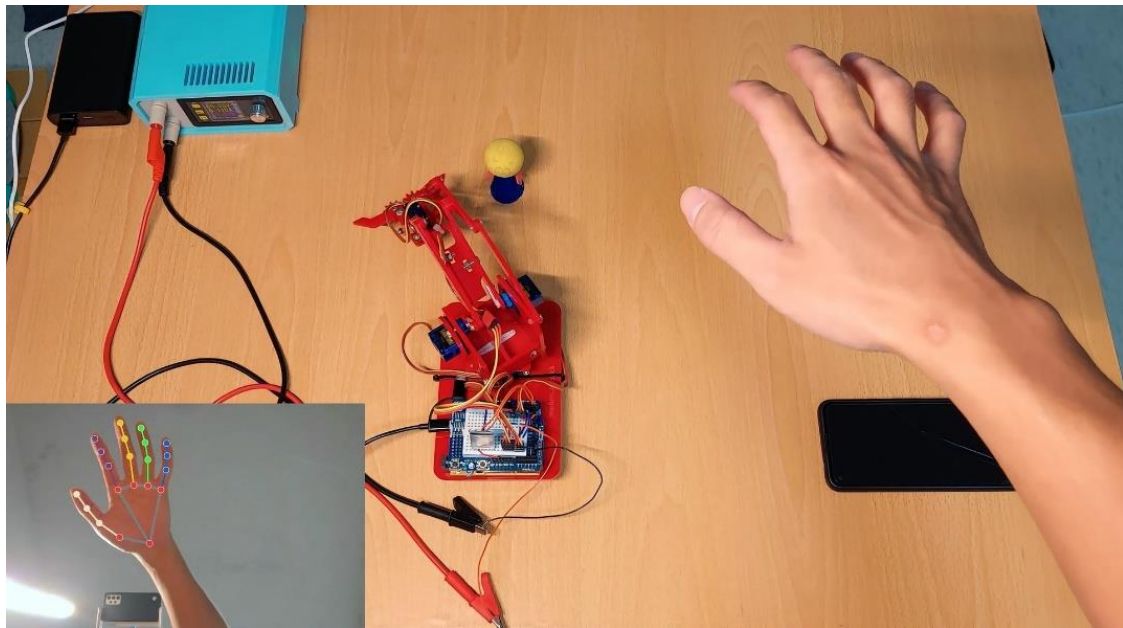
Where This Can Be Used

- **Assistive technology** — helping people with mobility impairments control robotic aids.
- **Industrial automation** — hands-free control in manufacturing or hazardous settings.
- **Healthcare** — touch-free interaction in sterile environments like operation theatres.
- **Education** — a low-cost, hands-on way for students to learn computer vision and robotics.
- **Smart homes** — gesture control for household robotic systems.
- **Disaster response** — remote gesture control for things like bomb disposal or rescue robots.

Results

Here's what I observed once the system was up and running:

- Open-palm and fist gestures were detected reliably under normal lighting, with minimal lag.
- The MeArm moved smoothly and tracked hand position across the X, Y and Z axes.
- The claw opened on an open palm and closed on a fist, as expected.
- Overall operation stayed stable without noticeable jitter or misalignment.
- Servo angle ranges and gesture thresholds could be tuned to fit different users.



The MeArm responding to hand gestures during testing

What I'd Add Next

- More gestures (like pinch or swipe) for finer control of individual joints.
- Switch to an ESP32 for onboard Wi-Fi control, so I don't need a laptop paired to it.
- Some visual or audio feedback to confirm which gesture was recognized.
- A lighter on-device ML model to make detection more robust in different lighting.

References

- Indonesian Journal of Electrical Engineering and Informatics (IJEI), Vol. 11, No. 3, September 2023, pp. 727–737, ISSN: 2089-3272, DOI: 10.52549/ijeei.v11i3.4491
- IET Conference Proceedings, Volume 2024, Issue 18 — <https://doi.org/10.1049/icp.2024.3947>

*Department of Electronics and Communication Engineering — SDM College of Engineering and Technology,
Dharwad-580002*